The **Add Two Numbers** problem on LeetCode involves adding two non-negative integers represented as **linked lists**, where the digits are stored in reverse order. Each node contains a single digit, and the task is to return the sum as a linked list.

Problem Statement

Given two non-empty linked lists:

- Each node represents a single digit.
- The digits are stored in reverse order (1's digit is the head of the list).
- Add the two numbers and return the result as a linked list.

Example:

Input:

```
l1 = [2,4,3]   // represents 342
l2 = [5,6,4]   // represents 465
```

Output:

```
[7,0,8]        // represents 807
```

Java Solution

```
class ListNode {
    int val;
    ListNode next;

    ListNode() {}
    ListNode(int val) {
        this.val = val;
    }
    ListNode(int val, ListNode next) {
        this.val = val;
        this.next = next;
    }
}

public class AddTwoNumbers {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        // Dummy head for the resultant linked list
        ListNode dummyHead = new ListNode(0);
        ListNode current = dummyHead;
        int carry = 0;

        // Traverse both linked lists
        while (l1 != null || l2 != null || carry != 0) {
            int x = (l1 != null) ? l1.val : 0;
            int y = (l2 != null) ? l2.val : 0;

            int sum = x + y + carry;
```

```
            carry = sum / 10;  // Calculate carry for next step
            current.next = new ListNode(sum % 10);  // Add the digit to the
result list

            // Move to the next nodes
            current = current.next;
            if (l1 != null) l1 = l1.next;
            if (l2 != null) l2 = l2.next;
        }

        return dummyHead.next; // Return the head of the resultant list
    }

    public static void main(String[] args) {
        // Example input
        ListNode l1 = new ListNode(2, new ListNode(4, new ListNode(3)));
        ListNode l2 = new ListNode(5, new ListNode(6, new ListNode(4)));

        AddTwoNumbers solution = new AddTwoNumbers();
        ListNode result = solution.addTwoNumbers(l1, l2);

        // Print result
        while (result != null) {
            System.out.print(result.val + " ");
            result = result.next;
        }
    }
}
```

Explanation

1. **Input as Linked Lists**:
   - The two numbers are represented as linked lists in reverse order.
   - `l1 = [2,4,3]` (342), `l2 = [5,6,4]` (465).
2. **Traverse Both Lists**:
   - Use a `while` loop to iterate through both lists.
   - Add corresponding nodes' values, and manage a `carry` if the sum exceeds 9.
3. **Handle Carry**:
   - If a carry remains after processing all nodes, add a new node to the result.
4. **Result Linked List**:
   - The result is built by appending nodes containing the sum's digits.

Example Walkthrough

Input:

```
l1 = [2,4,3]
l2 = [5,6,4]
```

Step-by-Step:

Step	l1.val	l2.val	Sum	Carry	Result List
1	2	5	7	0	[7]
2	4	6	10	1	[7, 0]
3	3	4	8	0	[7, 0, 8]

Output:

```
[7,0,8] (807)
```

Complexity

1. **Time Complexity**: O(max(m, n)), where m and n are the lengths of `l1` and `l2`.
2. **Space Complexity**: O(max(m, n)), for the resultant linked list.